

picoCTF Steganography Challenges - Complete Notes

Category: Forensics / Steganography

Tools Used: file, zsteg, strings, binwalk, exiftool, xdg-open, Python PIL

Table of Contents

- Challenge 1: extensions (flag.txt)
- Challenge 2: pico.flag.png
- Challenge 3: buildings.png
- Steganography Tools Reference
- Common Techniques
- Quick Diagnosis Flowchart
- Python LSB Extraction Script
- Key Takeaways

Challenge 1: extensions (flag.txt)

Challenge Description

“This is a really weird text file. Can you find the flag?”

Hints:

- How do operating systems know what kind of file it is? (It’s not just the ending!)
- Make sure to submit the flag as picoCTF{XXXXXX}

File

```
flag.txt
```

Step 1: Check Real File Type

```
file flag.txt
```

Output:

```
flag.txt: PNG image data, 1697 x 608, 8-bit/color RGB, non-interlaced
```

Key Insight: The file has a .txt extension but is actually a **PNG image**!

Step 2: Verify with Magic Bytes

```
xxd flag.txt | head -5
```

Output:

```
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG.....IHDR
```

Magic bytes 89 50 4E 47 = PNG signature

Step 3: Rename and Open

```
mv flag.txt flag.png
xdg-open flag.png
```

Result: The flag was **visible in the image itself!**

Flag

```
picoCTF{now_you_know_about_extensions}
```

Challenge 2: pico.flag.png

Challenge Description

“Download this image and find the flag.”

Hints:

- We know the end sequence of the message will be `$t3g0`.

File

```
pico.flag.png
```

Step 1: Check File Type

```
file pico.flag.png
```

Output:

```
pico.flag.png: PNG image data, 585 x 172, 8-bit/color RGBA, non-interlaced
```

Step 2: Run zsteg

```
zsteg pico.flag.png
```

Output:

```
b1,r,lsb,xy      .. text: "~__B>wV_G@"
b1,rgb,lsb,xy    .. text: "picoCTF{7h3r3_15_n0_5p00n_96ae0ac1}$t3g0"
b1,abgr,lsb,xy   .. text: "E2A5q4E%uSA"
b2,b,lsb,xy      .. text: "AAPAAQTAAA"
...
```

Key Insight: The flag is hidden in the **RGB channels' LSB** (`b1,rgb,lsb,xy`).

The hint `$t3g0` confirms this is **steganography** (stego)!

Flag

```
picoCTF{7h3r3_15_n0_5p00n_96ae0ac1}
```

Challenge 3: buildings.png

Challenge Description

“There’s something in the building. Can you retrieve the flag?”

Hints:

- There is data encoded somewhere... there might be an online decoder.

File

```
buildings.png
```

Step 1: Check File Type

```
file buildings.png
```

Output:

```
buildings.png: PNG image data, 657 x 438, 8-bit/color RGBA, non-interlaced
```

Step 2: Run zsteg

```
zsteg buildings.png
```

Output:

```
b1,r,lsb,xy      .. text: "^5>R5YZrG"
b1,rgb,lsb,xy    .. text: "picoCTF{h1d1ng_1n_th3_b1t5}"
b1,abgr,msb,xy   .. file: OpenPGP Secret Key
b2,b,lsb,xy      .. text: "XuH}p#8Iy="
...
```

Key Insight: Again, the flag is in `b1,rgb,lsb,xy` - RGB channels’ LSB.

Flag

```
picoCTF{h1d1ng_1n_th3_b1t5}
```

Steganography Tools Reference

Tool	Install	Purpose	Usage
zsteg	gem install zsteg	PNG/BMP LSB steganography	zsteg image.png
steghide	apt install steghide	Hide/extract data in images	steghide extract -sf img.jpg -p ""
stegseek	apt install stegseek	Brute force steghide passwords	stegseek image.jpg wordlist.txt
binwalk	apt install binwalk	Find hidden files/data	binwalk -e image.png

Table 1 – continued

Tool	Install	Purpose	Usage
exiftool	apt install libimage-exiftool-perl	Check metadata	exiftool image.png
strings	Built-in	Find printable strings	strings image.png
pngcheck	apt install pngcheck	PNG chunk analysis	pngcheck -v image.png

Online Tools

Tool	URL	Use For
StegOnline	https://stegonline.georgeom.net/	Bit plane analysis, LSB extraction
AperiSolve	https://www.aperisolve.com/	Multi-technique stego analysis
Steganography Online	https://stylesuxx.github.io/steganography/	LSB encode/decode

Common Techniques

1. Fake File Extensions

Extension	Actual Type	Detection
.txt	PNG/JPG	file command
.jpg	ZIP	file command
.png	ELF executable	file command

Always run **file** first!

2. LSB (Least Significant Bit) Steganography

How it works:

- Each pixel has RGB(A) values (0-255)
- The least significant bit of each channel can hide data
- Changes are invisible to the human eye

zsteg channels explained:

Channel Code	Meaning
b1	1 bit per channel
b2	2 bits per channel
b4	4 bits per channel
r,g,b,a	Red, Green, Blue, Alpha channels

Table 4 – continued

Channel Code	Meaning
rgb	All RGB channels combined
lsb	Least Significant Bit
msb	Most Significant Bit
xy	Pixel order (left-to-right, top-to-bottom)

3. Appended Data

Data hidden after the image’s IEND chunk:

```
# Check for appended data
binwalk image.png
tail -c 1000 image.png | xxd
```

4. Metadata Hiding

Hidden in EXIF or PNG chunks:

```
exiftool image.png
pngcheck -v image.png
```

Quick Diagnosis Flowchart

```
Get image file
|
v
file image.png          <-- Check real file type
|
+----> Not an image?    --> Check magic bytes, rename extension
|
+----> Is PNG/JPG?      --> Continue
|
v
zsteg image.png          <-- ALWAYS try this first for PNG
|
+----> Found flag?      --> Done!
|
+----> Nothing?         --> Try next steps
|
v
steghide extract -sf image.png -p ""
|
+----> Works?           --> Done!
|
+----> Needs password?  --> Try stegseek with wordlist
|
v
```

```
binwalk -e image.png
|
+---> Found files?    --> Check extracted files
|
v
exiftool image.png
|
+---> Hidden metadata? --> Done!
|
v
Python LSB extraction  <-- Manual method
|
v
Online decoders        <-- Last resort
(StegOnline, AperiSolve)
```

Python LSB Extraction Script

```
#!/usr/bin/env python3
# PNG LSB Steganography Extractor
# Extracts hidden data from PNG images using Least Significant Bits

from PIL import Image

def extract_lsb(image_path):
    # Extract LSB data from all channels
    img = Image.open(image_path)
    pixels = list(img.getdata())

    results = {}

    # Determine channels (RGB or RGBA)
    if len(pixels[0]) == 4:
        channels = [('R', 0), ('G', 1), ('B', 2), ('A', 3)]
    else:
        channels = [('R', 0), ('G', 1), ('B', 2)]

    for name, idx in channels:
        # Extract LSB from this channel
        binary = ''.join([str(p[idx] & 1) for p in pixels])

        # Convert binary to text
        chars = [binary[i:i+8] for i in range(0, len(binary), 8)]
        text = ''.join([chr(int(c, 2)) for c in chars if len(c) == 8])

        results[name] = text

    # Check for flag
    if 'pico' in text.lower() or 'flag' in text.lower():
```

```
        print(f"[!] Possible flag found in {name} channel!")

    return results

def main():
    import sys

    if len(sys.argv) != 2:
        print(f"Usage: {sys.argv[0]} <image.png>")
        sys.exit(1)

    image_path = sys.argv[1]
    print(f"[*] Analyzing: {image_path}")
    print("=" * 50)

    results = extract_lsb(image_path)

    for channel, text in results.items():
        print(f"
=== {channel} Channel ===")
        print(text[:500]) # Print first 500 chars
        print("...")

if __name__ == "__main__":
    main()
```

Usage:

```
python3 lsb_extract.py image.png
```

Key Takeaways

- 1. Always run **file** first – extensions can lie!
- 2. For PNG stego, always try **zsteg** first – it finds most CTF flags instantly
- 3. Look for **b1,rgb,lsb,xy** – this is the most common hiding spot
- 4. **Hints matter** – \$t3g0 = steganography, “online decoder” = try web tools
- 5. **Multiple techniques exist** – LSB, metadata, appended data, fake extensions
- 6. **When zsteg fails** – try steghide, binwalk, exiftool, or Python scripts

All Flags

Challenge	Flag
extensions (flag.txt)	picoCTF{now_you_know_about_extensions}
pico.flag.png	picoCTF{7h3r3_15_n0_5p00n_96ae0ac1}
buildings.png	picoCTF{h1d1ng_1n_th3_b1t5}

Resources

- [zsteg GitHub](#)
- [StegOnline](#)
- [AperiSolve](#)
- [Sleuthkit](#)

Happy Steganography Hunting! 🕵️